

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory work n. 1
of Operating Systems

The OS as a Resource Manager

Supervisers:

Dumitrașcu Marius, *University assistant*

Reițman Patricia, *University assistant*

DISA, FCIM, UTM

Chișinău – 2026

Table of Contents

1 Introduction **3**

2 Before we start **4**

3 Building the OS **8**

3.1 Build it and boot it 8

3.2 Use xv6 as the Unix it is 8

3.3 Read the source 8

3.4 Write first xv6 program 8

4 CONCLUSIONS **9**

1 Introduction

The goal of this laboratory work is to become familiar with xv6, a small Unix-like teaching operating system. xv6 is a modern reimplementation of Unix Version 6, containing the main components of a real operating system while remaining small enough for its source code to be studied directly.

The operating system is built from source and executed on a RISC-V machine provided by an emulator. This makes it possible to use xv6 as a small Unix system, inspect both its user programs and kernel code, and run a program written specifically for it.

2 Before we start

The work is performed in an Ubuntu virtual machine running in VirtualBox. The Ubuntu system itself runs on the ARM64 architecture, while xv6 is built for RISC-V.

RISC-V is an open instruction set architecture developed at the University of California, Berkeley. Its development began in 2010 as a new architecture for computer architecture research and processor design. The name refers to the fifth generation of the Berkeley RISC architecture lineage.

An instruction set architecture defines the machine instructions understood by a processor together with fundamental aspects of its programming model. RISC-V can therefore be implemented by physical processors as well as by software emulators. xv6 uses RISC-V as its target architecture, independently of the architecture on which the development tools themselves are running.

Since Ubuntu is running on ARM64 while the generated xv6 code must contain RISC-V instructions, a RISC-V cross-compilation toolchain is used. The `riscv64-linux-gnu-gcc` compiler translates C source files into object files containing 64-bit RISC-V machine code.

The separate object files are then combined by `riscv64-linux-gnu-ld`. The linker resolves references between them and produces the binaries that form the kernel and user programs. The linker, assembler, `objdump`, `objcopy`, and several related tools are provided by GNU Binutils for RISC-V.

The build sequence is controlled by GNU Make according to the rules stored in the xv6 `Makefile`. It invokes the required compilation and linking commands and provides targets for building and starting the system.

The resulting RISC-V machine code cannot be executed directly by the ARM64 processor used by Ubuntu. QEMU provides the required RISC-V system in software. The `qemu-system-riscv64` emulator represents a complete 64-bit RISC-V computer on which xv6 can boot and execute.

The execution environment therefore consists of several layers:

Physical computer $\rightarrow$ VirtualBox $\rightarrow$ Ubuntu ARM64 $\rightarrow$ QEMU RISC-V $\rightarrow$ xv6.

GDB Multiarch is also part of the development toolchain. It can debug programs for architectures other than that of the host machine and can be connected to an emulated RISC-V system to inspect execution, memory, processor registers, and kernel state.

The xv6 sources are expected to be stored in `~/xv6-riscv`. To begin, we will look for this directory in the home folder.

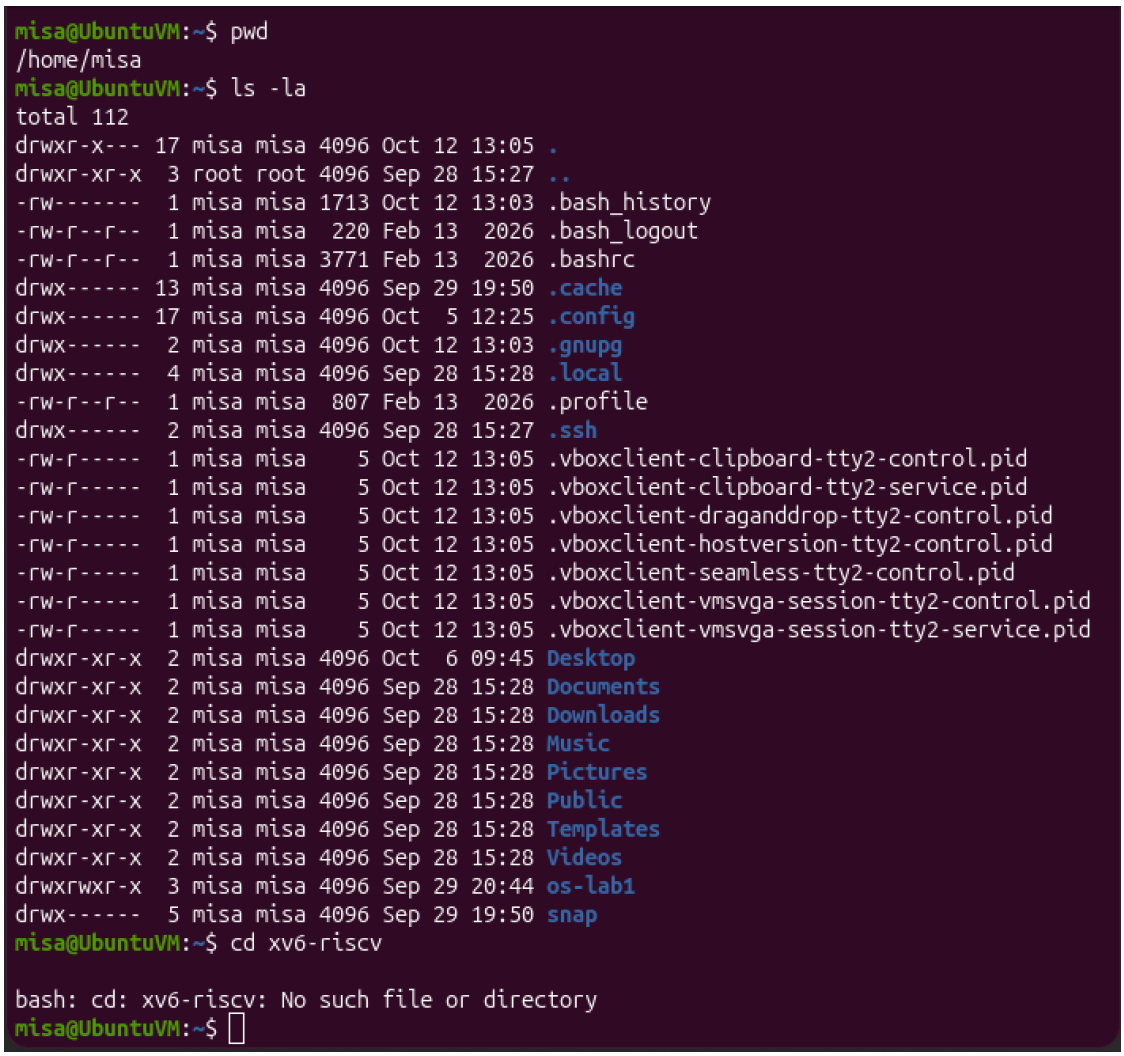


Figure 2.1 - Checking the home directory for the xv6 source code

The `xv6-riscv` directory is not present in the home folder, so the xv6 source code still needs to be obtained.

Before obtaining the source code, we check whether the tools required to build and run xv6 are available.

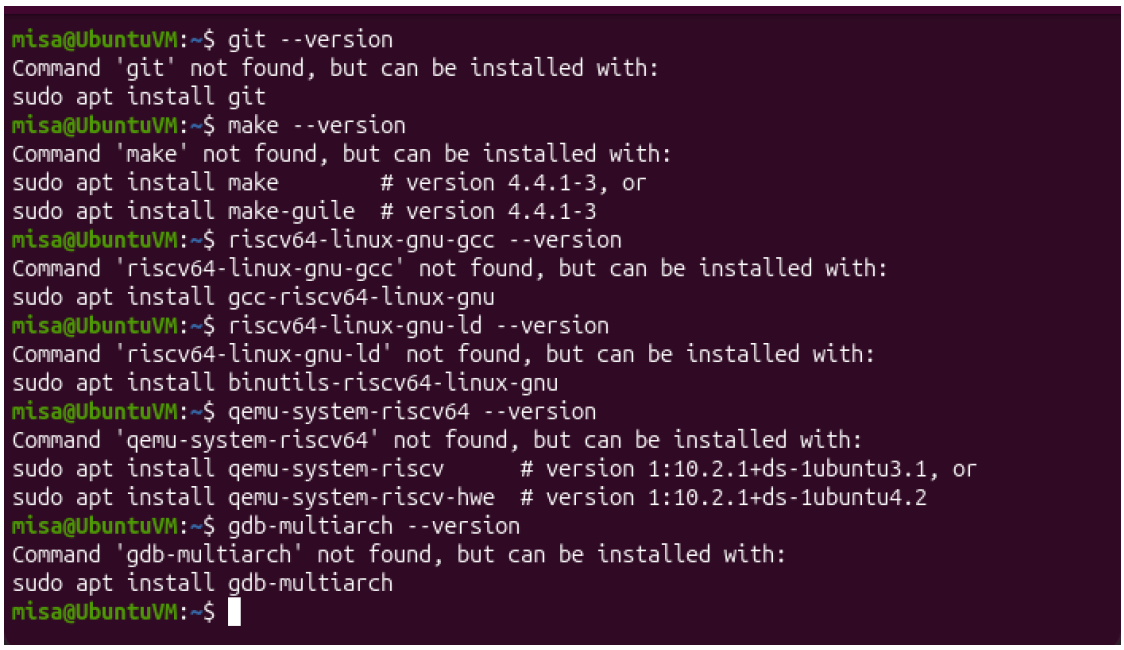


Figure 2.2 - Checking the availability of the required development tools

None of the required tools are currently available in the system. The development environment therefore needs to be prepared before xv6 can be built and executed.

The missing tools are installed through the Ubuntu package manager.

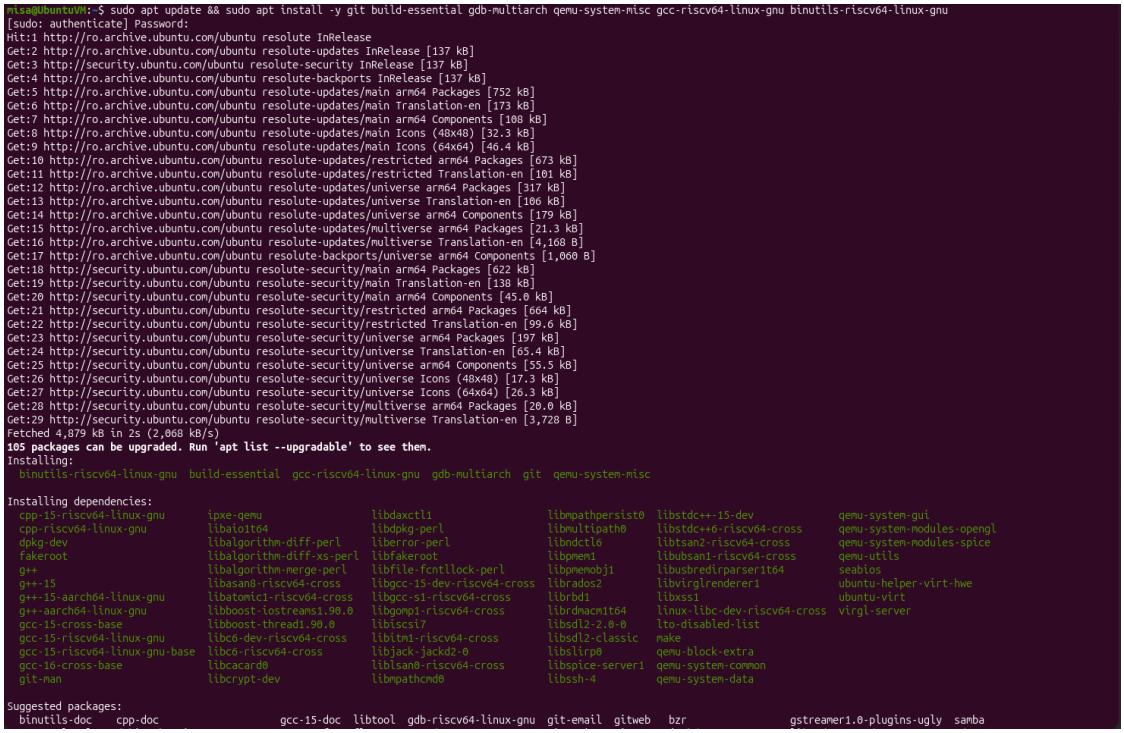


Figure 2.3 - Installing the xv6 development toolchain

The installed tools were then checked again.

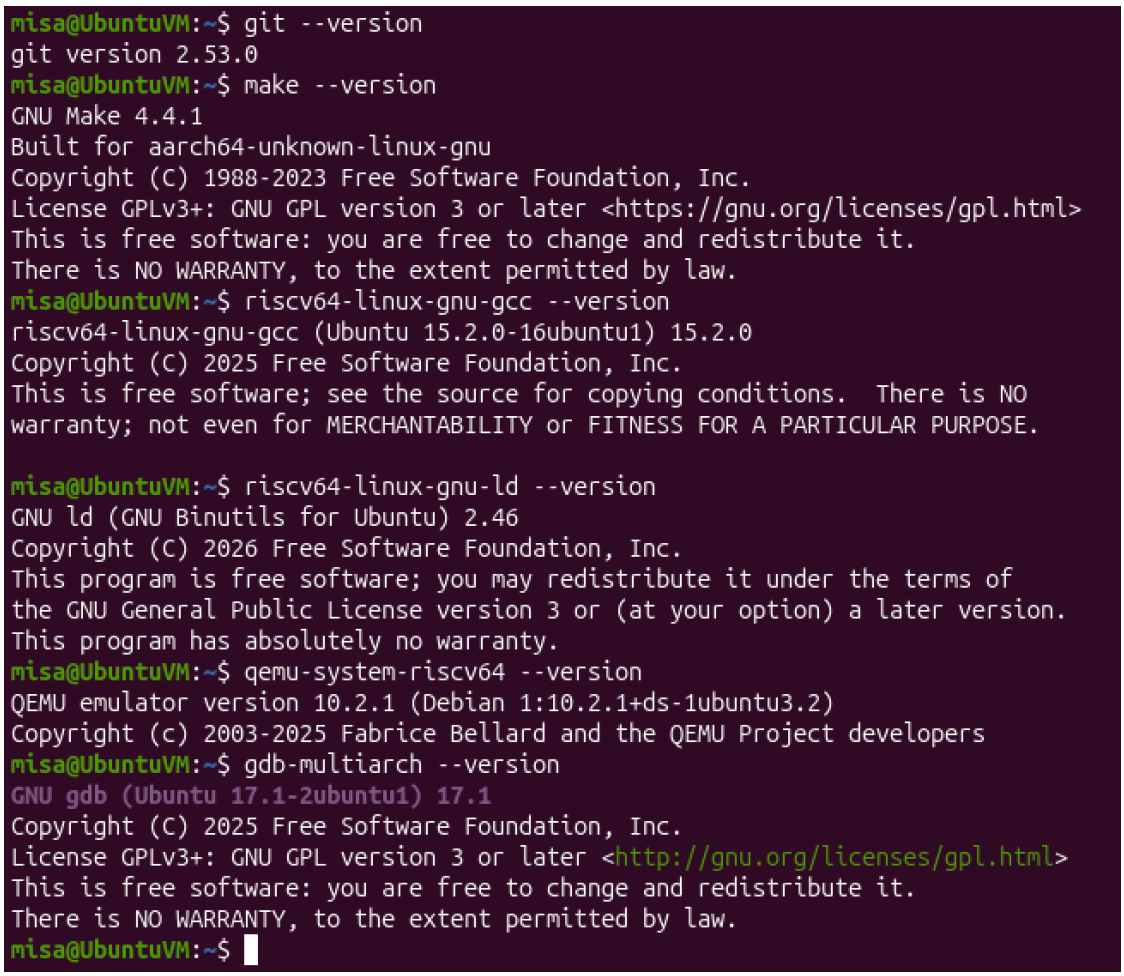


Figure 2.4 - Verification of the installed development tools

All required development tools are now installed and available in the system. The environment is ready for working with xv6.

With the development environment ready, the xv6 source code can now be obtained from the official repository.



Figure 2.5 - Cloning and inspecting the xv6 source repository

The xv6 source tree is now available in `~/xv6-riscv`. It contains the kernel sources, user programs,

the Makefile, and the other files required to build the system.

3 Building the OS

3.1 Build it and boot it

The xv6 kernel and user programs were built from source using the RISC-V toolchain. Make compiled and linked the required components, created the filesystem image, and launched the resulting system in QEMU.

```
misa@UbuntuVM:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nog
raphic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id
=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

Figure 3.1 - Booting xv6 in the QEMU RISC-V emulator

The boot completed successfully. After initializing the system, xv6 started its shell and presented the command prompt.

3.2 Use xv6 as the Unix it is

3.3 Read the source

3.4 Write first xv6 program

4 CONCLUSIONS